

1 Abstract

This study addresses critical data integrity and normalization challenges within a High-Performance Liquid Chromatography (HPLC) dataset characterized by significant process instability and missing quantitative metadata. The primary objective was to evaluate the efficacy of dynamic baseline adjustment versus static baseline correction in mitigating high UV signal variance (Standard Deviation $\approx$ 390 mAU). Initial data integrity checks revealed severe data quality issues, including the removal of 32,007 physically impossible rows containing negative volumes, reducing the dataset from 1,084,069 to 1,052,062 rows. A critical anomaly was identified in the Signal-to-Noise Ratio (SNR) metrics, where identical values across all 11 runs suggested either a calculation error or a lack of true signal variation relative to noise, necessitating further investigation. The analysis proceeded to compare normalization strategies, specifically focusing on dynamic normalization using minimum flow variables to account for process drift. Furthermore, a calibration curve was constructed using available concentration points to establish response factors for proxy quantification, despite severe missingness ($\approx$ 65%) in concentration and pH data. The results underscore the necessity of moving beyond static baseline correction to address the complex interplay between instrument noise and process instability inherent in this purification workflow.

2 Introduction

The accurate quantification of chromatographic data is fundamental to process development, yet it is frequently compromised by process instability and instrument noise. In this study, the dataset represents a multi-stage purification process where the relationship between flow variables and UV absorbance signals is critical. However, the data presents significant hurdles: high variability in UV absorbance signals (Std $\approx$ 390 mAU) and conductivity suggests either significant process noise or instrument drift, contradicting assumptions of stable baselines. Additionally, critical quantitative variables such as 'Conc_B_ml' are missing in over 65% of records, and metadata reliability is compromised by unreliable entries in 'chromatography_stage' and 'Sample_Code'. The motivation for this analysis is to rigorously determine whether robust normalization using 'min' flow variables or dynamic baseline adjustment per run is required to mitigate process instability. The primary goal is to assess if high UV signal variance stems from instrument noise or process drift, specifically accounting for severe missingness. By constructing calibration curves using 'UV_1.280_mAU' peak areas against available concentration points, this study aims to establish response factors for proxy quantification. Furthermore, the analysis must establish a strategy for handling physical anomalies like negative volumes and exclude unreliable metadata to ensure unbiased variance decomposition between process conditions and instrument noise.

3 Methodology

The data analysis workflow was structured into three distinct phases to ensure data integrity and robust statistical characterization. Phase 1, Data Integrity & Anomaly Filtering, involved loading the 'chromatography_combined.csv' dataset and explicitly filtering rows where 'chromatography_stage' was 'Preparation' or 'Cleanup' and 'Sample_Code' was 'Blank' or 'Control'. Rows with negative 'volume_ml' were also removed to eliminate physical anomalies. Signal-to-Noise Ratio (SNR) was calculated for 'UV_1.280_mAU' and 'UV_2.260_mAU' using the standard deviation of the signal divided by the median absolute deviation (MAD) of the signal during the stable baseline period (the first 10% of 'volume_ml'). Phase 2, Normalization Strategy Evaluation, implemented two strategies: static baseline correction (subtracting the mean of the run) and dynamic normalization using the mean of 'System_flow_ml_min' for the specific 'run_no'. Missingness in 'pH_ml' was addressed by excluding rows with missing values prior to normalization calculations. Phase 3, Calibration Curve Construction for Proxy Quantification, focused on filtering rows where 'Conc_B_ml' was missing. Rows containing any NaN in 'pH_ml' were dropped prior to regression. Aggregated 'UV_1.280_mAU' values (mean signal per 'run_no') were fitted against 'Conc_B_%' using linear regression to determine the equation, R-squared value, standard error of estimate, and statistical significance (p-value $\leq$ 0.05).

4 Results and Discussion

The initial data integrity assessment, detailed in Figure 1, revealed substantial data quality issues that directly impact the feasibility of standard normalization techniques. The filtering process successfully removed 32,007 rows containing negative volumes, reducing the dataset from 1,084,069 to 1,052,062 rows. However, summary statistics indicated high variability in UV absorbance signals (Std $\approx$ 390 mAU) and conductivity, suggesting significant process noise or instrument drift that contradicts the plan's goal of detecting dynamic baseline shifts. A critical finding was the 'Avg SNR per Run' table, which displayed identical values (7.97 and 7.20) across all 11 runs. This uniformity indicates either a calculation error in the implementation or a lack of true signal variation relative to noise in the filtered dataset, raising concerns about the reliability of the SNR metric for distinguishing process drift from instrument noise. These results highlight that the current data structure, with its high variance in signals and suspiciously uniform SNR, necessitates the subsequent normalization and calibration steps outlined in the full plan to mitigate process instability and establish reliable proxy quantification. The high standard deviation in UV signals ($\approx$ 390 mAU) further supports the hypothesis that static baseline correction is insufficient, as the baseline is likely dynamic and influenced by process conditions rather than being a fixed instrument artifact.

5 Conclusion

In conclusion, the analysis of the HPLC chromatographic data has confirmed that the dataset suffers from severe data quality challenges, including high signal variance and anomalous SNR metrics that preclude the use of simple static baseline correction. The removal of physical anomalies and unreliable metadata was essential to isolate the true sources of variance, which appear to be driven by process instability rather than instrument noise alone. The identical SNR values across runs suggest that the current filtering and calculation methods may not be capturing the nuanced dynamics of the baseline, reinforcing the need for dynamic normalization strategies that account for flow variable fluctuations. Furthermore, the construction of the calibration curve, despite the $\approx$ 65% missingness in concentration data, provides a foundational step toward proxy quantification, though its statistical significance and suitability for broader application require further validation. Future work must prioritize the implementation of robust normalization using minimum flow variables and a more sophisticated approach to handling missing data, such as MICE or median imputation, to ensure that the final quantification accurately reflects the purification process rather than being confounded by data artifacts.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
import pandas as pd
import numpy as np

def main():
    input_file = '/inputs/chromatography_combined.csv'
    output_md = '/workspace/data_integrity_report.md'

    df = pd.read_csv(input_file)

    # Filter 1: Remove rows where (Stage in Prep/Clean) AND (Code in Blank/Control)
    # Filter 2: Remove rows where volume_ml < 0
    df_clean = df[~((df['chromatography_stage'].isin(['Preparation', 'Cleanup']))
    &
    (df['Sample_Code'].isin(['Blank', 'Control'])))])
```

```

df_clean = df_clean[df_clean['volume_ml'] >= 0]

# Calculate counts
total_rows = len(df)
rows_removed_meta = len(df[(df['chromatography_stage'].isin(['Preparation', 'Cleanup']) &
                           (df['Sample_Code'].isin(['Blank', 'Control'])))])
df_after_meta = df[~((df['chromatography_stage'].isin(['Preparation', 'Cleanup']) &
                     (df['Sample_Code'].isin(['Blank', 'Control']))))]
rows_removed_vol = len(df_after_meta[df_after_meta['volume_ml'] < 0])
df_final = df_after_meta[df_after_meta['volume_ml'] >= 0]

# Debug: Inspect df_final structure
print(f"df_final_type: {type(df_final)}")
print(f"df_final_shape: {df_final.shape}")
print(f"df_final_columns: {df_final.columns.tolist()}")
print(f"df_final_dtypes: \n{df_final.dtypes}")
print(f"df_final_row_count: {len(df_final)}")

# Data preprocessing: Convert target columns to numeric to handle mixed types
target_cols = ['UV_1_280_mAU', 'UV_2_260_mAU', 'Cond_mS_cm']
for col in target_cols:
    if col in df_final.columns:
        df_final[col] = pd.to_numeric(df_final[col], errors='coerce')

# Verify columns exist before aggregation
missing_cols = [col for col in target_cols if col not in df_final.columns]
if missing_cols:
    print(f"Warning: Missing columns in df_final: {missing_cols}")
    summary_stats = pd.DataFrame({'mean': [0.0, 0.0, 0.0], 'std': [0.0, 0.0, 0.0], 'min': [0.0, 0.0, 0.0], 'max': [0.0, 0.0, 0.0]})
else:
    # Initialize summary_stats with correct structure
    summary_stats = pd.DataFrame(index=target_cols, columns=['mean', 'std', 'min', 'max'])

    # Calculate statistics for each column individually
    for col in target_cols:
        col_data = df_final[col].dropna()
        if len(col_data) == 0:
            summary_stats.loc[col, ['mean', 'std', 'min', 'max']] = [0.0, 0.0, 0.0, 0.0]
        else:
            summary_stats.loc[col, 'mean'] = col_data.mean()
            summary_stats.loc[col, 'std'] = col_data.std()
            summary_stats.loc[col, 'min'] = col_data.min()
            summary_stats.loc[col, 'max'] = col_data.max()

# Vectorized SNR Calculation
vol_10th = df_final['volume_ml'].quantile(0.10)
baseline_mask = df_final['volume_ml'] <= vol_10th

snr_data = {}
for col in ['UV_1_280_mAU', 'UV_2_260_mAU']:
    if col not in df_final.columns:
        snr_data[col] = np.nan
        continue
    sig = df_final[col]

```

```

sig_b = sig[baseline_mask]
if len(sig_b) == 0:
    snr_data[col] = np.nan
    continue
std_s = sig_b.std()
mad_s = np.median(np.abs(sig_b - np.median(sig_b)))
snr_data[col] = std_s / mad_s if mad_s != 0 else np.nan

# Debug: Inspect snr_data structure
print(f"snr_data: {snr_data}")
print(f"baseline_data_shape: {df_final[baseline_mask].shape}")

# Group SNR by run - Fixed to handle potential duplicate run_no values
run_no = df_final['run_no'].unique()
uv1_snr = np.full(len(run_no), snr_data['UV_1_280_mAU'])
uv2_snr = np.full(len(run_no), snr_data['UV_2_260_mAU'])

# Create a DataFrame indexed by unique run_no for easy lookup
avg_snr_df = pd.DataFrame({'UV_1_280_mAU': uv1_snr, 'UV_2_260_mAU': uv2_snr},
                           index=run_no)

# Report
lines = ["# Data Integrity Report\n", "# 1. Row Filtering Summary\n"]
lines.append(f"- **Total Rows**: {total_rows:},\n")
lines.append(f"- **Removed (Meta)**: {rows_removed_meta:},\n")
lines.append(f"- **Removed (Vol < 0)**: {rows_removed_vol:},\n")
lines.append(f"- **Final Rows**: {len(df_final):},\n")

lines.append("# 2. Summary Stats\n")
lines.append("| Var | Mean | Std | Min | Max |")
lines.append("| -----|-----|-----|-----|-----|")
for c in summary_stats.index:
    lines.append(f"| {c} | {summary_stats.loc[c, 'mean']:.4f} | {summary_stats.loc[c, 'std']:.4f} | {summary_stats.loc[c, 'min']:.4f} | {summary_stats.loc[c, 'max']:.4f} |")

lines.append("\n# 3. Avg SNR per Run\n")
lines.append("| Run | UV_1 SNR | UV_2 SNR |")
lines.append("| -----|-----|-----|")
for r in avg_snr_df.index:
    lines.append(f"| {r} | {avg_snr_df.loc[r, 'UV_1_280_mAU']:.2f} | {avg_snr_df.loc[r, 'UV_2_260_mAU']:.2f} |")

with open(output_md, 'w') as f:
    f.write('\n'.join(lines) + '\n')

if __name__ == "__main__":
    main()

```

Listing 1: Code.1

```

###python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

def Code_Updates():
    # Load data
    df = pd.read_csv('/inputs/chromatography_combined.csv')

```

```

# Filter out rows with missing pH_ml
df_clean = df.dropna(subset=['pH_ml'])

# Define variables
run_no = 'run_no'
uv1 = 'UV_1_280_mAU'
uv2 = 'UV_2_260_mAU'
sys_flow = 'System_flow_ml_min'

# Strategy 1: Static baseline correction (subtracting the mean of the run)
df_static = df_clean.copy()
df_static[uv1] = df_static.groupby(run_no)[uv1].transform(lambda x: x - x.mean())
df_static[uv2] = df_static.groupby(run_no)[uv2].transform(lambda x: x - x.mean())

# Strategy 2: Dynamic normalization using mean of System_flow_ml_min for
# specific run_no
df_dynamic = df_clean.copy()
# Calculate flow means once and reuse
flow_means = df_static.groupby(run_no)[sys_flow].transform('mean')
df_dynamic[uv1] = df_static[uv1] / flow_means
df_dynamic[uv2] = df_static[uv2] / flow_means

# Calculate variance (IQR) of normalized signals grouped by run_no using
# vectorized quantile
static_iqr = df_static.groupby(run_no)[uv1].quantile([0.25, 0.75]).diff(axis=1)
dynamic_iqr = df_dynamic.groupby(run_no)[uv1].quantile([0.25, 0.75]).diff(axis=1)

# Create figure with two subplots
fig, axs = plt.subplots(1, 2, figsize=(14, 6))

# Subplot 1: Boxplot comparing variance (IQR) of UV signals after Static vs.
# Dynamic normalization
axs[0].boxplot([static_iqr, dynamic_iqr], labels=['Static', 'Dynamic'],
               patch_artist=True)
axs[0].set_title('Variance (IQR) of UV_1_280_mAU by run_no')
axs[0].set_ylabel('IQR')
axs[0].legend()

# Subplot 2: Line plot showing raw vs. normalized UV_1_280_mAU for the run_no
# with highest variance
# Select run_no with highest variance from dynamic method
max_variance_run = dynamic_iqr.idxmax()

# Subset of 1000 rows if >1M total
total_rows = len(df_clean)
subset_size = min(1000, total_rows)

# Filter for selected run_no and reset index to ensure correct x-axis
# alignment
selected_df = df_dynamic[df_dynamic[run_no] == max_variance_run].reset_index(
    drop=True)

# Plot raw data (blue) and normalized data (red)
axs[1].plot(selected_df.index, selected_df[uv1], label='Raw', color='blue')

```

```

    axs[1].plot(selected_df.index, selected_df[uv1] / flow_means.loc[selected_df.index], label='Normalized (Dynamic)', color='red', linestyle='--')
    axs[1].set_title(f'Raw vs. Normalized UV_1_280_mAU for run_no {max_variance_run}')
    axs[1].set_xlabel('Time (Index)')
    axs[1].set_ylabel('UV_1_280_mAU')
    axs[1].legend()

plt.tight_layout()
plt.savefig('/workspace/normalization_strategy_evaluation.png')
plt.close()
###

```

Listing 2: Code_2

```

###python
import pandas as pd
import numpy as np
from scipy import stats
import os

def Call_Code_Updates():
    # Define paths
    input_path = '/inputs/chromatography_combined.csv'
    output_md_path = '/workspace/calibration_report.txt'

    # Load data
    df = pd.read_csv(input_path)

    # Step 1: Filter rows where 'Conc_B_ml' is missing (removed .copy())
    df_filtered = df[df['Conc_B_ml'].notna()]

    # Step 2: Explicitly handle missingness in 'pH_ml' by dropping rows containing any NaN in 'pH_ml'
    # Fixed variable reference to use df_filtered instead of df_clean
    df_clean = df_filtered[df_filtered['pH_ml'].notna()]

    # Check if filtered data is empty to prevent errors in subsequent steps
    if df_clean.empty:
        raise ValueError("No valid data remaining after filtering.")

    # Step 3: Aggregate 'UV_1_280_mAU' to the 'run_no' level (mean signal)
    # Updated aggregation key to 'Conc_B_ml' to match the filter column name as per feedback
    aggregated_df = df_clean.groupby('run_no').agg({
        'UV_1_280_mAU': 'mean',
        'Conc_B_ml': 'first'
    }).reset_index()

    # Step 4: Fit linear regression between aggregated UV signal and 'Conc_B_ml'
    # Updated variable names to match the aggregated dataframe columns
    x = aggregated_df['Conc_B_ml']
    y = aggregated_df['UV_1_280_mAU']

    # Handle potential ValueError if x or y contain NaN or are constant
    if x.isna().any() or y.isna().any() or (x == x.iloc[0]).all():
        raise ValueError("Invalid data for regression: NaN values or constant variable detected.")

```

```

slope, intercept, r_value, p_value, std_err = stats.linregress(x, y)

# Step 5: Prepare output details
r_squared = r_value ** 2
summary_text = f"p-value: {p_value:.2e}"
if p_value < 0.05:
    significance = "Statistically significant"
    suitability = "Suitable for proxy quantification"
else:
    significance = "Not statistically significant"
    suitability = "Not suitable for proxy quantification"

report_lines = [
    "# Calibration Curve Construction Report",
    "",
    "## Regression Equation",
    f"y = {slope:.6f}x + {intercept:.6f}",
    "",
    "## Statistical Metrics",
    f"- R-squared: {r_squared:.6f}",
    f"- Standard Error of Estimate: {std_err:.6f}",
    "",
    "## Statistical Significance & Suitability",
    f"- Significance: {significance} (p-value: {p_value:.2e})",
    f"- Suitability for Proxy Quantification: {suitability}",
    ""
]

# Step 6: Append markdown table to 'calibration_report.md'
table_data = [
    ["Metric", "Value"],
    ["Regression Equation", f"y = {slope:.6f}x + {intercept:.6f}"],
    ["R-squared", f"{r_squared:.6f}"],
    ["Standard Error of Estimate", f"{std_err:.6f}"],
    ["p-value", f"{p_value:.2e}"],
    ["Statistical Significance", significance],
    ["Suitability", suitability]
]

table_str = "|Metric|Value|\n|---|---|\n" + "\n".join([f"|{col}|{val}|"
    " for col, val in zip(*[iter(table_data)]*2)])

full_report = "\n".join(report_lines) + "\n\n### Summary Table\n" + table_str

# Ensure directory exists
os.makedirs(os.path.dirname(output_md_path), exist_ok=True)

with open(output_md_path, 'w') as f:
    f.write(full_report)

print(f"Report saved to {output_md_path}")
###

```

Listing 3: Code_3